

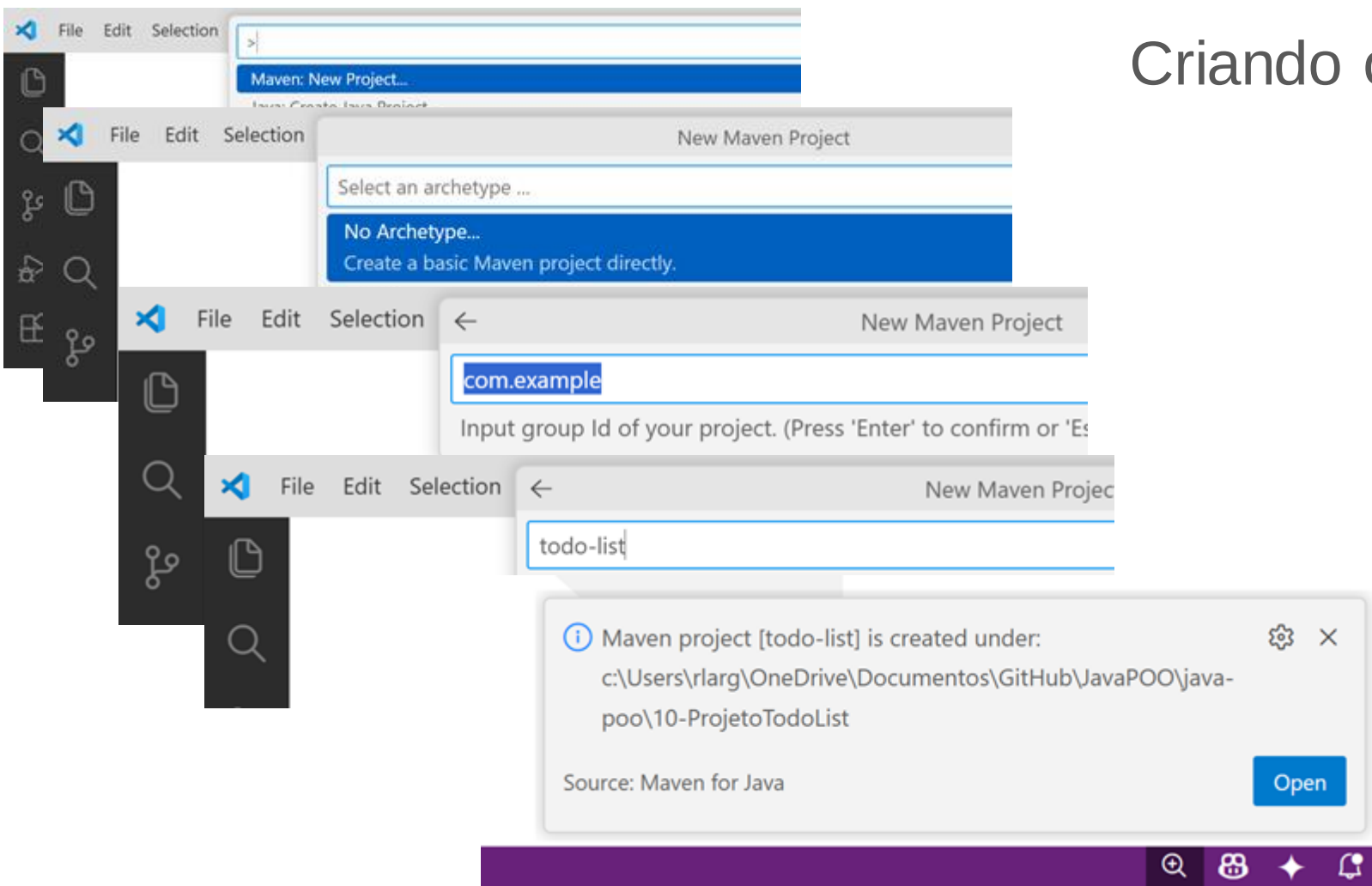
Aula 14 – 26/05/2026

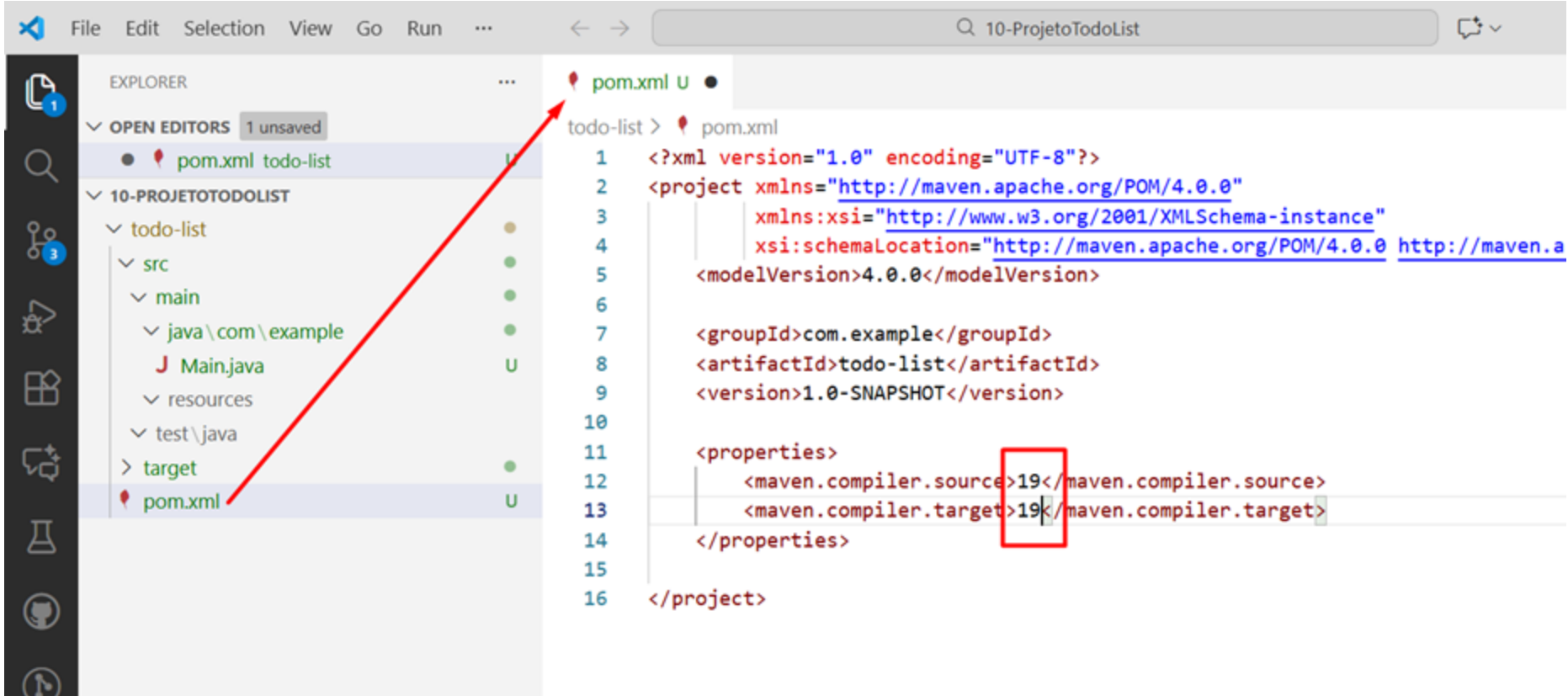
Projeto: To-do list

Sumário

- Aplicação prática fullstack:
 - Fazer um CRUD de um TODO list no sqlite
 - Backend:
 - Tabelas para usuários e tarefas
 - Implementar controle de acesso por token
 - Papéis: ADMIN e USER, com enum
 - Frontend:
 - Habilitar arquivos estáticos no Javalin para publicar assets e páginas HTML
 - Implementar uma interface dinâmica utilizando [alpine.js](#) e estilizado com bootstrap. O app deve ser um SPA com menu simples de navegação entre:
 - Quadro de lista de tarefas, com título e botão para marcar como feita e remover
 - Admin (restrito ao papel ADMIN) com a tabela de usuários (com form de inclusão e comando de exclusão) e a lista de tarefas de todos os usuários

Criando o projeto





Confira a versão do JDK

File Edit Selection View Go Run ... 10-ProjetoToDoList

EXPLORER

OPEN EDITORS

- ✕ J Main.java todo-list\src\main\java\co... U

10-PROJETOTODOLIST

- > .vscode
- ▼ todo-list
 - ▼ src
 - ▼ main
 - ▼ java\com\example
 - J Main.java U
 - ▼ resources
 - ▼ test\java
 - > target
 - 📌 pom.xml U

J Main.java U ✕

todo-list > src > main > java > com > example > J Main.java > Main

```
1 package com.example;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(x: "Hello world!");
6     }
7 }
```

Run | Debug

OUTPUT DEBUG CONSOLE PORTS GITLENS

▼ TERMINAL

```
PS C:\Users\rlarg\OneDrive\Documentos\GitHub\JavaP00\java-poo\1
> & 'C:\Program Files\Java\openjdk-19.0.2\bin\java.exe' '-XX:+
ExceptionMessages' '-cp' 'C:\Users\rlarg\OneDrive\Documentos\Gi
-poo\10-ProjetoToDoList\todo-list\target\classes' 'com.example.
Hello world!
○ PS C:\Users\rlarg\OneDrive\Documentos\GitHub\JavaP00\java-poo\1
>
```

Teste

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
4      <modelVersion>4.0.0</modelVersion>
5
6      <groupId>com.example</groupId>
7      <artifactId>todo-list</artifactId>
8      <version>1.0-SNAPSHOT</version>
9
10 > <properties> ...
13 </properties>
14
15 <dependencies>
16     <!-- https://mvnrepository.com/artifact/io.javalin/javalin-bundle -->
17     <dependency>
18         <groupId>io.javalin</groupId>
19         <artifactId>javalin-bundle</artifactId>
20         <version>6.7.0</version>
21     </dependency>
22     <!-- https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc -->
23     <dependency>
24         <groupId>org.xerial</groupId>
25         <artifactId>sqlite-jdbc</artifactId>
26         <version>3.51.0.0</version>
27     </dependency>
28     <!-- https://mvnrepository.com/artifact/org.mindrot/jbcrypt -->
29     <dependency>
30         <groupId>org.mindrot</groupId>
31         <artifactId>jbcrypt</artifactId>
32         <version>0.4</version>
33     </dependency>
34 </dependencies>
35
36 </project>
```

Inclua as dependências

Teste o web server

The image shows a development environment with a code editor on the left and a browser window on the right. The code editor displays a Java file named `Main.java` with the following content:

```
1 package com.example;
2
3 import io.javalin.Javalin;
4
5 public class Main {
6     public static void main(String[] args) {
7         Javalin app = Javalin.create(config -> {
8             // Define o Content-Type padrão para todas as respostas
9             config.http.defaultContentType = "text/plain; charset=UTF-8";
10
11             // Define codificação de caracteres
12             config.pvt.javaLangErrorHandler((res, error) -> {
13                 res.setCharacterEncoding("UTF-8");
14             });
15         }).start(7070);
16         app.get("/", ctx -> {
17             // Teste
18             ctx.result("Hello, World!");
19         });
20     }
21 }
```

The browser window, titled `localhost:7070`, displays the text `Hello, World!`. A red arrow points from the `ctx.result("Hello, World!");` line in the code editor to the browser window.

Below the code editor, the `OUTPUT` and `DEBUG CONSOLE` tabs are visible. The `DEBUG CONSOLE` tab shows the following output:

```
PS C:\Users\rlarg\OneDrive\Documentos\GitHub\JavaPoo\java-poo\10-Projet...
jdk-19.0.2\bin\java.exe - '@C:\Users\rlarg\AppData\Local\Temp\cp_7sc3lpe...
Main'
```

The `TERMINAL` tab shows the following output:

```
https://javalin.io/documentation

[main] INFO io.javalin.Javalin - Javalin start
[main] INFO io.javalin.Javalin - Listening on http://localhost:7070/
[main] INFO io.javalin.Javalin - You are running Javalin 6.3.0 (released June 22, 2025). Your Javalin version is
```



File Edit Selection View Go Run Terminal Help



EXPLORER

OPEN EDITORS

× J Main.java todo-list\src\main\java\co... U

10-PROJETOTODOLIST

> .vscode

todo-list

src

main

java\com\example

J Main.java

resources

test\java

target

pom.xml

J Main.java U ×

todo-list > src > main > java > com > e

1 package com.example;

2

New File...

New Folder...

Reveal in File Explorer

Shift+Alt+R

Open in Integrated Terminal

Add Folder to Java Source Path

Remove Folder from Java Source Path

New Java File



New Java Package...

com.example.model

Press 'Enter' to confirm your input or 'Escape' to cancel

src
main
java\com\exam
model
Main.java
resources
test\java
target
pom.xml

Reveal in File Explorer

Shift+Alt+R

Open in Integrated Terminal

Add Folder to Java Source Path

Remove Folder from Java Source Path

New Java File



New Java Package...

New Java Project...

Modernize Java

```
out.println(x: "Hello world");
```

Class...

Interface...

Enum...

Role

Press 'Enter' to confirm your input or 'Escape' to cancel

Main.java U

Role.java U X

todo-list > src > main > java > com > example > model > Role.java > Role > ADMIN

```
1 package com.example.model;
```

```
2
```

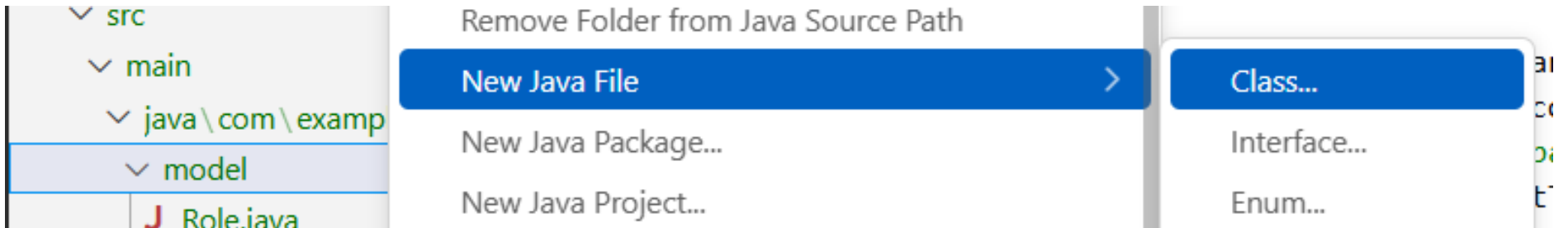
```
3 public enum Role {
```

```
4     USER, ADMIN
```

```
5 }
```

```
6
```

Vamos criar as classes User e Task



EXPLORER

OPEN EDITORS

- User.java todo-list/src/m... U

10-PROJETOTODOLIST

- .vscode
- todo-list
 - src
 - main
 - java\com\example
 - model
 - Role.java U
 - User.java U
 - Main.java U
 - resources
 - test\java
 - target
 - pom.xml U

User.java U X

todo-list > src > main > java > com > example > model > User.java > User > token

```
1 package com.example.model;
2
3 public class User {
4     private Integer id;
5     private String username;
6     private String password;
7     private Role role;
8     private String token;
9
10    public User() {}
11
12    public User(String username, String password, Role role) {
13        this.username = username;
14        this.password = password;
15        this.role = role;
16    }
17
18    // Getters e setters para todos os campos
19    public Integer getId() { return id; }
20    public void setId(Integer id) { this.id = id; }
21    public String getUsername() { return username; }
22    public void setUsername(String username) { this.username = username; }
23    public String getPassword() { return password; }
24    public void setPassword(String password) { this.password = password; }
25    public Role getRole() { return role; }
26    public void setRole(Role role) { this.role = role; }
27    public String getToken() { return token; }
28    public void setToken(String token) { this.token = token; }
29
30    @Override
31    public String toString() {
32        return "User{username='" + username + "', role=" + role + "}";
33    }
34 }
```

OPEN EDITORS

J User.java todo-list\src\m... U

X J Task.java todo-list\src\m... U

10-PROJETOTODOLIST

> .vscode ●

▼ todo-list ●

▼ src ●

▼ main ●

▼ java\com\example ●

▼ model ●

J Role.java U

J Task.java U

J User.java U

J Main.java U

▼ resources

▼ test\java

> target ●

pom.xml U

J User.java U

J Task.java U X

todo-list > src > main > java > com > example > model > J Task.java > Task > userId

```
1 package com.example.model;
2
3 public class Task {
4     private Integer id;
5     private String title;
6     private boolean done;
7     private Integer userId; // FK para User
8
9     public Task() {}
10
11     public Task(String title, boolean done, Integer userId) {
12         this.title = title;
13         this.done = done;
14         this.userId = userId;
15     }
16
17     // Getters e setters
18     public Integer getId() { return id; }
19     public void setId(Integer id) { this.id = id; }
20     public String getTitle() { return title; }
21     public void setTitle(String title) { this.title = title; }
22     public boolean isDone() { return done; }
23     public void setDone(boolean done) { this.done = done; }
24     public Integer getUserId() { return userId; }
25     public void setUserId(Integer userId) { this.userId = userId; }
26
27     @Override
28     public String toString() {
29         return "Task{title='" + title + "', done=" + done + "}";
30     }
31 }
```

✓ 10-PROJETOTODOLIST

> .vscode

✓ todo-list

✓ src

✓ main

✓ java \ com \ example

> m

J M

✓ res

✓ test

> target

📍 pom.xml

Add Folder to Java Source Path

Remove Folder from Java Source Pat

New Java File

New Java Package...

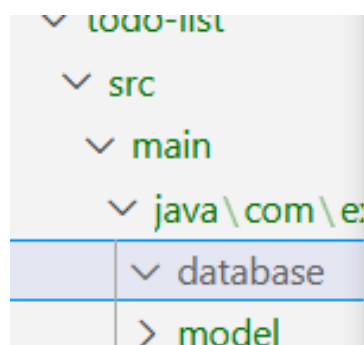
New Java Project

com.example.database

Press 'Enter' to confirm your input or 'Escape' to cancel

Find in Folder...

S



Remove Folder from Java Source Path

New Java File

done ;

Class...

er

SQLiteConnection

java

Press 'Enter' to confirm your input or 'Escape' to cancel

The Integer class wraps

EXPLORER

OPEN EDITORS

- ✕ J SQLiteConnection.java t... U

10-PROJETOTODOLIST

- > .vscode
- ▼ todo-list
 - ▼ src
 - ▼ main
 - ▼ java\com\example
 - ▼ database
 - J SQLiteConnection.java U
 - > model
 - J Main.java U
 - ▼ resources
 - ▼ test\java
 - > target
 - ▼ pom.xml U
 - ▼ tasks.db U

> OUTLINE

> TIMELINE

> JAVA PROJECTS

J SQLiteConnection.java U X

todo-list > src > main > java > com > example > database > J SQLiteConnection.java > SQLiteConnection

```
1 package com.example.database;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.Statement;
6
7 public class SQLiteConnection {
8     // String de conexão (cria o arquivo tasks.db na raiz do projeto)
9     private static final String URL = "jdbc:sqlite:tasks.db";
10
11     public static Connection connect() throws Exception {
12         return DriverManager.getConnection(URL);
13     }
14
15     // Método para criar as tabelas se elas não existirem
16     public static void createTables() {
17         String sqlUsers = "CREATE TABLE IF NOT EXISTS users (" +
18             "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
19             "username TEXT UNIQUE NOT NULL, " +
20             "password TEXT NOT NULL, " +
21             "role TEXT NOT NULL DEFAULT 'USER', " +
22             "token TEXT)";
23         String sqlTasks = "CREATE TABLE IF NOT EXISTS tasks (" +
24             "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
25             "title TEXT NOT NULL, " +
26             "done BOOLEAN DEFAULT FALSE, " +
27             "userId INTEGER NOT NULL, " +
28             "FOREIGN KEY (userId) REFERENCES users(id))";
29
30         try (Connection conn = connect(); Statement stmt = conn.createStatement()) {
31             stmt.execute(sqlUsers);
32             stmt.execute(sqlTasks);
33             System.out.println(x: "Tabelas criadas/verif.");
34         } catch (Exception e) {
35             e.printStackTrace();
36         }
37     }
38 }
```

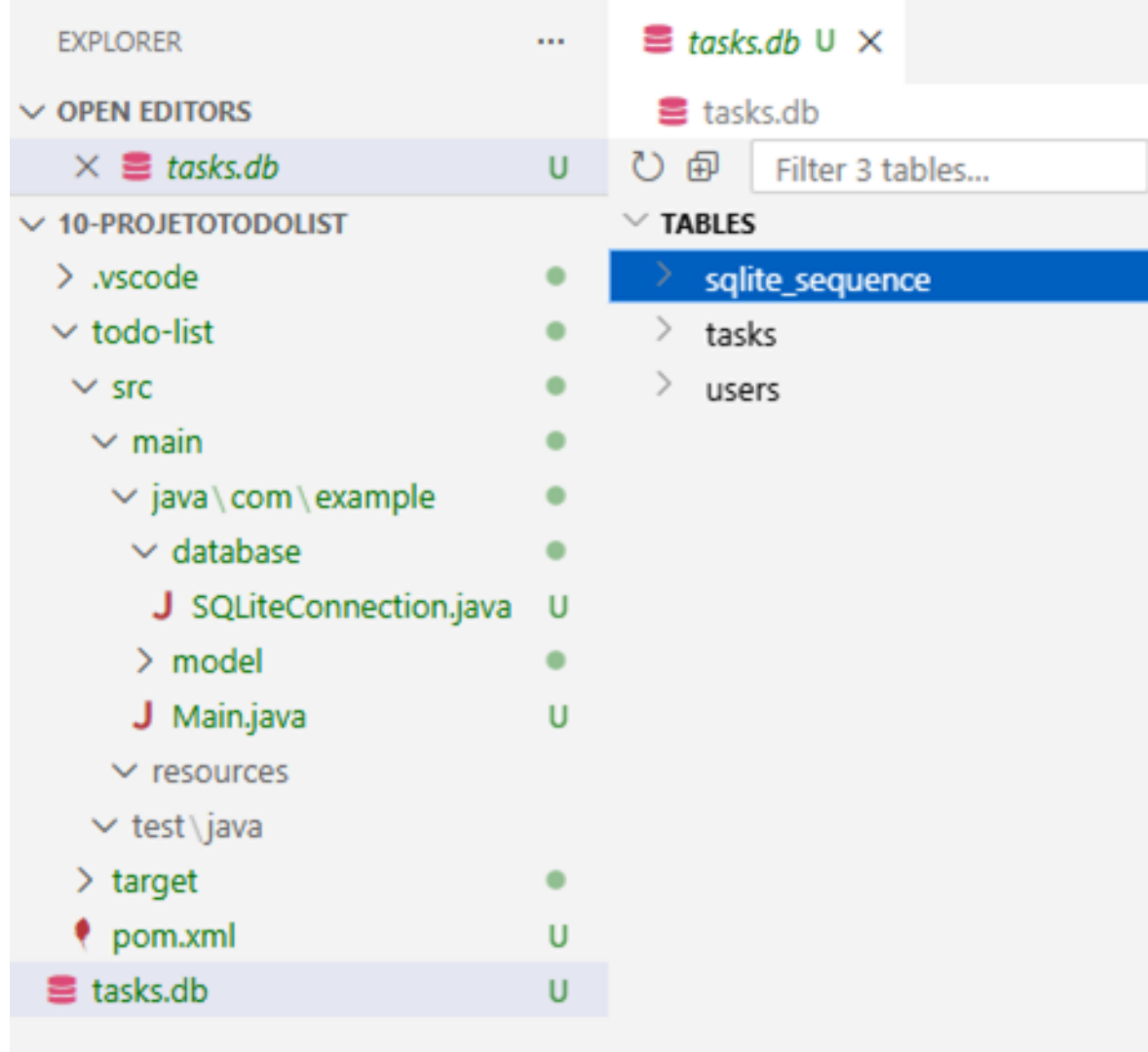
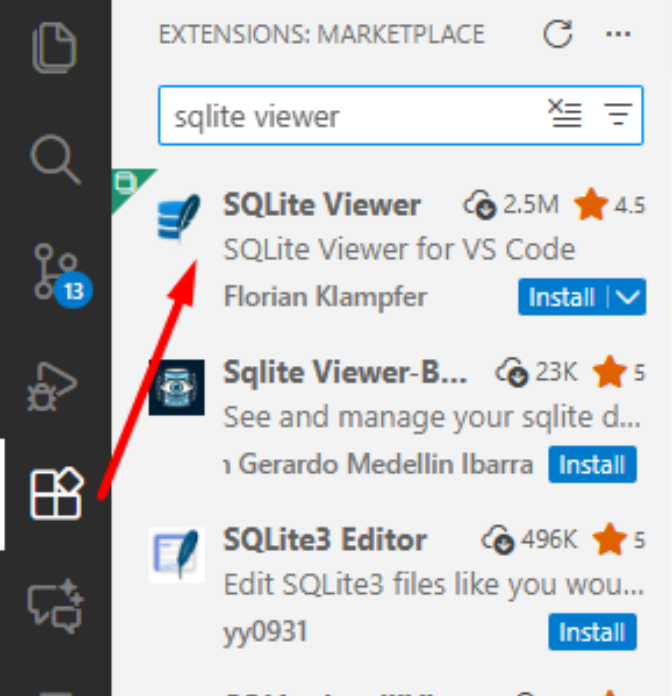

todo-list > src > main > java > com > example > J Main.java > Main > main(String

```
1 package com.example;
2
3 import com.example.database.SQLiteConnection;
4
5 import io.javalin.Javalin;
6
7 public class Main {
8     public static void main(String[] args) {
9         // 1. Garante que as tabelas existem ao iniciar
10        SQLiteConnection.createTables();
11
12        // 2. Inicia o servidor Javalin na porta 7070
13        Javalin app = Javalin.create(config -> {
14            app.get("/", ctx -> {
15
16            }
17        });
18    }
19 }
```

OUTPUT DEBUG CONSOLE PORTS GITLENS

TERMINAL

PS C:\Users\rlarg\OneDrive\Documentos\GitHub\JavaP00\java-poo\10-Proj
.0.2\bin\java.exe' '@C:\Users\rlarg\AppData\Local\Temp\cp_7sc3lpelhz
Tabelas criadas/verif.
[main] INFO io.javalin.Javalin - Starting Javalin ...



Para testar o database

▼ main

▼ java \ com \ example

> database

> model

🔴 Main.java

▼ resources

▼ test \ java

Open in Integrated Terminal

Add Folder to Java Source Path

Remove Folder from Java Source Path

New Java File

New Java Package...

com.example.repository

Press 'Enter' to confirm your input or 'Escape' to cancel



Vamos criar as classes
userRepository e TaskRepository

```
1 package com.example.repository;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.util.ArrayList;
7 import java.util.List;
8 import org.mindrot.jbcrypt.BCrypt;
9
10 import com.example.database.SQLiteConnection;
11 import com.example.model.Role;
12 import com.example.model.User;
13
14 public class UserRepository {
15
16 > /** ...
21 > public static void add(User user) throws Exception { ...
35
36 > /** ...
42 > public static boolean delete(String username) throws Exception { ...
51
52 > /** ...
59 > public static boolean validate(String username, String password) throws Exception { ...
73
74 > /** ...
80 > public static User getByUsername(String username) throws Exception { ...
98
99 > /** ...
105 > public static void updateToken(String username, String token) throws Exception { ...
114
115 > /** ...
120 > public static void removeToken(String token) throws Exception { ...
128
129 > /** ...
135 > public static User getUserByToken(String token) throws Exception { ...
151
152 > /** ...
157 > public static List<User> getAllUsers() throws Exception { ...
173 }
```

```

16  /**
17   * Insere um novo usuário no banco de dados (com senha hashada).
18   * @param user Objeto usuário preenchido (username, password, role)
19   * @throws Exception Se houver erro de SQL ou duplicação de username
20   */
21  public static void add(User user) throws Exception {
22      // Hashea a senha antes de salvar
23      String hashedPassword = BCrypt.hashpw(user.getPassword(), BCrypt.gensalt());
24      user.setPassword(hashedPassword); // Atualiza o objeto temporariamente
25
26      String sql = "INSERT INTO users(username, password, role) VALUES(?, ?, ?)";
27      try (Connection conn = SQLiteConnection.connect();
28          PreparedStatement pstmt = conn.prepareStatement(sql)) {
29          pstmt.setString(parameterIndex: 1, user.getUsername());
30          pstmt.setString(parameterIndex: 2, hashedPassword); // Usa o hash
31          pstmt.setString(parameterIndex: 3, user.getRole().name());
32          pstmt.executeUpdate();
33      }
34  }
35
36  /**
37   * Remove um usuário pelo nome de usuário.
38   * @param username Nome do usuário a ser removido
39   * @return true se apagou, false se não encontrou
40   * @throws Exception Erros de conexão
41   */
42  public static boolean delete(String username) throws Exception {
43      String sql = "DELETE FROM users WHERE username = ?";
44      try (Connection conn = SQLiteConnection.connect();
45          PreparedStatement pstmt = conn.prepareStatement(sql)) {
46          pstmt.setString(parameterIndex: 1, username);
47          int affectedRows = pstmt.executeUpdate();
48          return affectedRows > 0;
49      }
50  }

```

```
52  /**
53   * Valida se o par username/senha existe no banco (compara hashes).
54   * @param username Nome do usuário
55   * @param password Senha em texto plano (será hasheada pra comparação)
56   * @return true se as credenciais forem válidas
57   * @throws Exception Erros de conexão
58   */
59  public static boolean validate(String username, String password) throws Exception {
60      String sql = "SELECT password FROM users WHERE username = ?";
61      try (Connection conn = SQLiteConnection.connect();
62           PreparedStatement pstmt = conn.prepareStatement(sql)) {
63          pstmt.setString(parameterIndex: 1, username);
64          ResultSet rs = pstmt.executeQuery();
65          if (rs.next()) {
66              String hashedPassword = rs.getString(columnLabel: "password");
67              // Compara a senha entrada com o hash armazenado
68              return BCrypt.checkpw(password, hashedPassword);
69          }
70      }
71      return false;
72  }
```

```
74  /**
75   * Busca um usuário pelo nome de usuário (útil para Sessão).
76   * @param username Nome do usuário
77   * @return User ou null se não encontrado
78   * @throws Exception Erros de conexão
79   */
80  public static User getByUsername(String username) throws Exception {
81      String sql = "SELECT * FROM users WHERE username = ?";
82      User user = null;
83      try (Connection conn = SQLiteConnection.connect();
84           PreparedStatement pstmt = conn.prepareStatement(sql)) {
85          pstmt.setString(parameterIndex: 1, username);
86          ResultSet rs = pstmt.executeQuery();
87          if (rs.next()) {
88              user = new User();
89              user.setId(rs.getInt(columnLabel: "id"));
90              user.setUsername(rs.getString(columnLabel: "username"));
91              user.setPassword(rs.getString(columnLabel: "password"));
92              user.setRole(Role.valueOf(rs.getString(columnLabel: "role"))); // Converte string pra enum
93              user.setToken(rs.getString(columnLabel: "token"));
94          }
95      }
96      return user;
97  }
```



```

99  /**
100  * Atualiza o token de um usuário (usado no login).
101  * @param username Nome do usuário
102  * @param token Token gerado
103  * @throws Exception Erros de conexão
104  */
105  public static void updateToken(String username, String token) throws Exception {
106      String sql = "UPDATE users SET token = ? WHERE username = ?";
107      try (Connection conn = SQLiteConnection.connect();
108           PreparedStatement pstmt = conn.prepareStatement(sql)) {
109          pstmt.setString(parameterIndex: 1, token);
110          pstmt.setString(parameterIndex: 2, username);
111          pstmt.executeUpdate();
112      }
113  }
114
115  /**
116  * Limpa o token de um usuário.
117  * @param token Token gerado
118  * @throws Exception Erros de conexão
119  */
120  public static void removeToken(String token) throws Exception {
121      String sql = "UPDATE users SET token = '' WHERE token = ?";
122      try (Connection conn = SQLiteConnection.connect();
123           PreparedStatement pstmt = conn.prepareStatement(sql)) {
124          pstmt.setString(parameterIndex: 1, token);
125          pstmt.executeUpdate();
126      }
127  }

```

```
129  /**
130   * Busca um usuário pelo token (validação de acesso).
131   * @param token Token do usuário
132   * @return User ou null se inválido
133   * @throws Exception Erros de conexão
134   */
135  public static User getUserByToken(String token) throws Exception {
136      String sql = "SELECT * FROM users WHERE token = ?";
137      try (Connection conn = SQLiteConnection.connect();
138           PreparedStatement pstmt = conn.prepareStatement(sql)) {
139          pstmt.setString(parameterIndex: 1, token);
140          ResultSet rs = pstmt.executeQuery();
141          if (rs.next()) {
142              User u = new User();
143              u.setId(rs.getInt(columnLabel: "id"));
144              u.setUsername(rs.getString(columnLabel: "username"));
145              u.setRole(Role.valueOf(rs.getString(columnLabel: "role")));
146              return u;
147          }
148      }
149      return null; // Token inválido ou não encontrado
150  }
```

```
152  /**
153   * Lista todos os usuários (para admin).
154   * @return Lista de User (sem senha)
155   * @throws Exception Erros de conexão
156   */
157  public static List<User> getAllUsers() throws Exception {
158      String sql = "SELECT id, username, role FROM users"; // Sem senha por segurança
159      List<User> users = new ArrayList<>();
160      try (Connection conn = SQLiteConnection.connect();
161           PreparedStatement pstmt = conn.prepareStatement(sql);
162           ResultSet rs = pstmt.executeQuery()) {
163          while (rs.next()) {
164              User u = new User();
165              u.setId(rs.getInt(columnLabel: "id"));
166              u.setUsername(rs.getString(columnLabel: "username"));
167              u.setRole(Role.valueOf(rs.getString(columnLabel: "role")));
168              users.add(u);
169          }
170      }
171      return users;
172  }
```

OPEN EDITORS

X J TaskRepository.java tod... U

10-PROJETOTODOLIST

> .vscode

todo-list

src

main

java\com\example

database

J SQLiteConnection.java U

model

J Role.java U

J Task.java U

J User.java U

repository

J TaskRepository.java U

J UserRepository.java U

resources

test\java

> target

pom.xml U

tasks.db U

J TaskRepository.java U X

todo-list > src > main > java > com > example > repository > J TaskRepository.java > TaskRepository

```
1 package com.example.repository;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 import com.example.database.SQLiteConnection;
10 import com.example.model.Task;
11
12 public class TaskRepository {
13
14 > /** ...
19 > public static void add(Task task) throws Exception { ...
29
30 > /** ...
36 > public static List<Task> getByUserId(int userId) throws Exception { ...
54
55 > /** ...
60 > public static List<Task> getAll() throws Exception { ...
77
78 > /** ...
83 > public static void update(Task task) throws Exception { ...
93
94 > /** ...
100 > public static boolean delete(int id) throws Exception { ...
109 }
```

```
14  /**
15   * Insere uma nova tarefa no banco de dados.
16   * @param task Objeto tarefa preenchido (title, done, userId)
17   * @throws Exception Se houver erro de SQL
18   */
19  public static void add(Task task) throws Exception {
20      String sql = "INSERT INTO tasks(title, done, userId) VALUES(?, ?, ?)";
21      try (Connection conn = SQLiteConnection.connect();
22           PreparedStatement pstmt = conn.prepareStatement(sql)) {
23          pstmt.setString(parameterIndex: 1, task.getTitle());
24          pstmt.setBoolean(parameterIndex: 2, task.isDone());
25          pstmt.setInt(parameterIndex: 3, task.getUserId());
26          pstmt.executeUpdate();
27      }
28  }
```

```
30  /**
31   * Lista tarefas de um usuário específico.
32   * @param userId ID do usuário
33   * @return Lista de Task
34   * @throws Exception Erros de conexão
35   */
36  public static List<Task> getUserId(int userId) throws Exception {
37      String sql = "SELECT * FROM tasks WHERE userId = ?";
38      List<Task> tasks = new ArrayList<>();
39      try (Connection conn = SQLiteConnection.connect();
40           PreparedStatement pstmt = conn.prepareStatement(sql)) {
41          pstmt.setInt(parameterIndex: 1, userId);
42          ResultSet rs = pstmt.executeQuery();
43          while (rs.next()) {
44              Task t = new Task();
45              t.setId(rs.getInt(columnLabel: "id"));
46              t.setTitle(rs.getString(columnLabel: "title"));
47              t.setDone(rs.getBoolean(columnLabel: "done"));
48              t.setUserId(rs.getInt(columnLabel: "userId"));
49              tasks.add(t);
50          }
51      }
52      return tasks;
53  }
```

```
55  /**
56   * Lista todas as tarefas (para admin).
57   * @return Lista de Task
58   * @throws Exception Erros de conexão
59   */
60  public static List<Task> getAll() throws Exception {
61      String sql = "SELECT * FROM tasks";
62      List<Task> tasks = new ArrayList<>();
63      try (Connection conn = SQLiteConnection.connect();
64           PreparedStatement pstmt = conn.prepareStatement(sql);
65           ResultSet rs = pstmt.executeQuery()) {
66          while (rs.next()) {
67              Task t = new Task();
68              t.setId(rs.getInt(columnLabel: "id"));
69              t.setTitle(rs.getString(columnLabel: "title"));
70              t.setDone(rs.getBoolean(columnLabel: "done"));
71              t.setUserId(rs.getInt(columnLabel: "userId"));
72              tasks.add(t);
73          }
74      }
75      return tasks;
76  }
```

```
78     /**
79      * Atualiza uma tarefa (title e done).
80      * @param task Objeto tarefa com id preenchido
81      * @throws Exception Erros de conexão
82      */
83     public static void update(Task task) throws Exception {
84         String sql = "UPDATE tasks SET title = ?, done = ? WHERE id = ?";
85         try (Connection conn = SQLiteConnection.connect();
86             PreparedStatement pstmt = conn.prepareStatement(sql)) {
87             pstmt.setString(parameterIndex: 1, task.getTitle());
88             pstmt.setBoolean(parameterIndex: 2, task.isDone());
89             pstmt.setInt(parameterIndex: 3, task.getId());
90             pstmt.executeUpdate();
91         }
92     }
```



```
94      /**
95       * Remove uma tarefa por id.
96       * @param id ID da tarefa
97       * @return true se removeu, false se não encontrou
98       * @throws Exception Erros de conexão
99       */
100     public static boolean delete(int id) throws Exception {
101         String sql = "DELETE FROM tasks WHERE id = ?";
102         try (Connection conn = SQLiteConnection.connect();
103              PreparedStatement pstmt = conn.prepareStatement(sql)) {
104             pstmt.setInt(parameterIndex: 1, id);
105             int affectedRows = pstmt.executeUpdate();
106             return affectedRows > 0;
107         }
108     }
```

EXPLORER

10-PROJETOTODOLIST

- src
 - main
 - java\com\example
 - database
 - SQLiteConnection.java
 - model
 - Role.java
 - Task.java
 - User.java
 - repository
 - TaskRepository.java
 - UserRepository.java

Main.java

```
package com.example;

import com.example.database.SQLiteConnection;
import com.example.model.Role;
import com.example.model.Task;
import com.example.model.User;
import com.example.repository.TaskRepository;
import com.example.repository.UserRepository;
import io.javalin.Javalin;
import java.util.List;
import java.util.Map;
import java.util.UUID;

public class Main {

    public static void main(String[] args) {
        SQLiteConnection.createTables();

        Javalin app = Javalin.create(config -> { ...

        app.get("/", ctx -> ctx.result("API rodando!"));

        app.before("/api/*", ctx -> { ...
        app.before("/api/admin/*", ctx -> { ...
        app.post("/login", ctx -> { ...
        app.put("/logout", ctx -> { ...
        app.get("/api/admin/users", ctx -> { ...
        app.post("/api/admin/users", ctx -> { ...
        app.delete("/api/admin/users/{username}", ctx -> { ...
        app.get("/api/admin/tasks", ctx -> { ...
        app.get("/api/tasks", ctx -> { ...
        app.post("/api/tasks", ctx -> { ...
        app.put("/api/tasks/{id}", ctx -> { ...
        app.delete("/api/tasks/{id}", ctx -> { ...

        System.out.println("Servidor rodando em http://localhost:7078");
    }
}
```

Configurando as rotas do servidor web

```
16 SQLiteConnection.createTables();
17
18 Javalin app = Javalin.create(config -> {
19     // Define o Content-Type padrão para todas as respostas
20     config.http.defaultContentType = "text/plain; charset=UTF-8";
21
22     // Se estiver usando arquivos estáticos ou templates, verifique também:
23     config.pvt.javaLangErrorHandler((res, error) -> {
24         res.setCharacterEncoding("UTF-8");
25     });
26 }).start(7078);
27
28 app.get("/", ctx -> ctx.result("API rodando!"));
```

```
30 app.before("/api/*", ctx -> {
31     try {
32         String token = ctx.header("Authorization");
33         if (token == null || token.isEmpty()) {
34             ctx.status(401).json(Map.of("error", "Token necessário"));
35             ctx.skipRemainingHandlers();
36             return;
37         }
38         User user = UserRepository.getUserByToken(token);
39         if (user == null) {
40             ctx.status(401).json(Map.of("error", "Token inválido"));
41             ctx.skipRemainingHandlers();
42             return;
43         }
44         ctx.attribute("user", user);
45     } catch (Exception e) {
46         ctx.status(500).json(Map.of("error", "Erro interno: " + e.getMessage()));
47         ctx.skipRemainingHandlers();
48     }
49 });
50 app.before("/api/admin/*", ctx -> {
51     User user = ctx.attribute("user");
52     if (user == null) {
53         ctx.status(401).json(Map.of("error", "Usuário não autenticado"));
54         ctx.skipRemainingHandlers();
55         return;
56     }
57     if (user.getRole() != Role.ADMIN) {
58         ctx.status(403).json(Map.of("error", "Acesso negado"));
59         ctx.skipRemainingHandlers();
60         return;
61     }
62 });
```

```
63 app.post("/login", ctx -> {
64     try {
65         User loginData = ctx.bodyAsClass(User.class);
66         boolean valid = UserRepository.validate(loginData.getUsername(), loginData.getPassword());
67         if (!valid) {
68             ctx.status(401).json(Map.of("error", "Credenciais inválidas"));
69             return;
70         }
71         String token = UUID.randomUUID().toString();
72         UserRepository.updateToken(loginData.getUsername(), token);
73         ctx.json(Map.of("token", token));
74     } catch (Exception e) {
75         ctx.status(500).json(Map.of("error", "Erro interno: " + e.getMessage()));
76     }
77 });
78 app.put("/logout", ctx -> {
79     try {
80         String token = ctx.header("Authorization");
81         if (token != null) {
82             UserRepository.removeToken(token);
83         }
84         ctx.json(Map.of("message", "Logout OK"));
85     } catch (Exception e) {
86         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
87     }
88 });
```

```
89 app.get("/api/admin/users", ctx -> {
90     try {
91         List<User> users = UserRepository.getAllUsers();
92         ctx.json(users);
93     } catch (Exception e) {
94         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
95     }
96 });
97 app.post("/api/admin/users", ctx -> {
98     try {
99         User newUser = ctx.bodyAsClass(User.class);
100         UserRepository.add(newUser);
101         ctx.status(201).json(Map.of("message", "Usuário criado"));
102     } catch (Exception e) {
103         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
104     }
105 });
106 app.delete("/api/admin/users/{username}", ctx -> {
107     try {
108         String username = ctx.pathParam("username");
109         boolean deleted = UserRepository.delete(username);
110         if (deleted) {
111             ctx.json(Map.of("message", "Usuário deletado"));
112         } else {
113             ctx.status(404).json(Map.of("error", "Usuário não encontrado"));
114         }
115     } catch (Exception e) {
116         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
117     }
118 });
```

```
119 app.get("/api/admin/tasks", ctx -> {
120     try {
121         List<Task> tasks = TaskRepository.getAll();
122         ctx.json(tasks);
123     } catch (Exception e) {
124         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
125     }
126 });
127 app.get("/api/tasks", ctx -> {
128     try {
129         User user = ctx.attribute("user");
130         if (user == null) {
131             ctx.status(401).json(Map.of("error", "Usuário não autenticado"));
132             return;
133         }
134         List<Task> tasks = TaskRepository.getByUserId(user.getId());
135         ctx.json(tasks);
136     } catch (Exception e) {
137         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
138     }
139 });
```

```
140 app.post("/api/tasks", ctx -> {
141     try {
142         User user = ctx.attribute("user");
143         if (user == null) {
144             ctx.status(401).json(Map.of("error", "Usuário não autenticado"));
145             return;
146         }
147         Task newTask = ctx.bodyAsClass(Task.class);
148         newTask.setUserId(user.getId());
149         newTask.setDone(false);
150         TaskRepository.add(newTask);
151         ctx.status(201).json(Map.of("message", "Tarefa criada"));
152     } catch (Exception e) {
153         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
154     }
155 });
```



```
156 app.put("/api/tasks/{id}", ctx -> {
157     try {
158         User user = ctx.attribute("user");
159         if (user == null) {
160             ctx.status(401).json(Map.of("error", "Usuário não autenticado"));
161             return;
162         }
163         int taskId = Integer.parseInt(ctx.pathParam("id"));
164         Task updateTask = ctx.bodyAsClass(Task.class);
165         updateTask.setId(taskId);
166         List<Task> userTasks = TaskRepository.getByUserId(user.getId());
167         boolean owns = userTasks.stream().anyMatch(t -> t.getId().equals(taskId));
168         if (!owns) {
169             ctx.status(403).json(Map.of("error", "Tarefa não pertence ao usuário"));
170             return;
171         }
172         TaskRepository.update(updateTask);
173         ctx.json(Map.of("message", "Tarefa atualizada"));
174     } catch (Exception e) {
175         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
176     }
177 });
```

```
178 app.delete("/api/tasks/{id}", ctx -> {
179     try {
180         User user = ctx.attribute("user");
181         if (user == null) {
182             ctx.status(401).json(Map.of("error", "Usuário não autenticado"));
183             return;
184         }
185         int taskId = Integer.parseInt(ctx.pathParam("id"));
186         List<Task> userTasks = TaskRepository.getByUserId(user.getId());
187         boolean owns = userTasks.stream().anyMatch(t -> t.getId().equals(taskId));
188         if (!owns) {
189             ctx.status(403).json(Map.of("error", "Tarefa não pertence ao usuário"));
190             return;
191         }
192         boolean deleted = TaskRepository.delete(taskId);
193         if (deleted) {
194             ctx.json(Map.of("message", "Tarefa deletada"));
195         } else {
196             ctx.status(404).json(Map.of("error", "Tarefa não encontrada"));
197         }
198     } catch (Exception e) {
199         ctx.status(500).json(Map.of("error", "Erro: " + e.getMessage()));
200     }
201 });
202
203 System.out.println("Servidor rodando em http://localhost:7078");
204 }
```

Para testar, cria um admin padrão se a tabela de usuários estiver vazia

```
13 public class Main {  
    Run | Debug  
14 public static void main(String[] args) {  
15     SQLiteConnection.createTables();  
16  
17     // Cria admin padrão se não houver usuários  
18     try {  
19         List<User> users = UserRepository.getAllUsers();  
20         if (users.isEmpty()) {  
21             User admin = new User("admin", "admin123", Role.ADMIN);  
22             UserRepository.add(admin);  
23             System.out.println("Admin padrão criado: username=admin, password=admin123");  
24         }  
25     } catch (Exception e) {  
26         System.err.println("Erro ao criar admin padrão: " + e.getMessage());  
27     }  
28  
29 > Javalin app = Javalin.create(config -> { ...  
--
```

EXPLORER

MAIN.java U tasks.db U X

OPEN EDITORS

Main.java todo-list/src/vm... U

tasks.db U

10-PROJETOTODOLIST

.vscode

todo-list

src

main

java\com\example

database

SQLiteConnection.java U

model

Role.java U

Task.java U

User.java U

repository

TaskRepository.java U

UserRepository.java U

Main.java U

resources

test\java

Teste.java U

target

pom.xml U

tasks.db U

Filter 3 tables...

Rows: 1

TABLES

sqlite_sequence

tasks

users

id	username	password	role	token
1	admin	\$2a\$10\$mzSLTvS6aT6kOTt58JNGxe8Xm.s7tpY3arRu5V...	ADMIN	NULL
2				

SQLITE VIEWER FREE v0.10.6 Page 1 / 1

Try Hydejack, a Jekyll theme for hackers.

OUTPUT DEBUG CONSOLE PORTS GITLENS

TERMINAL

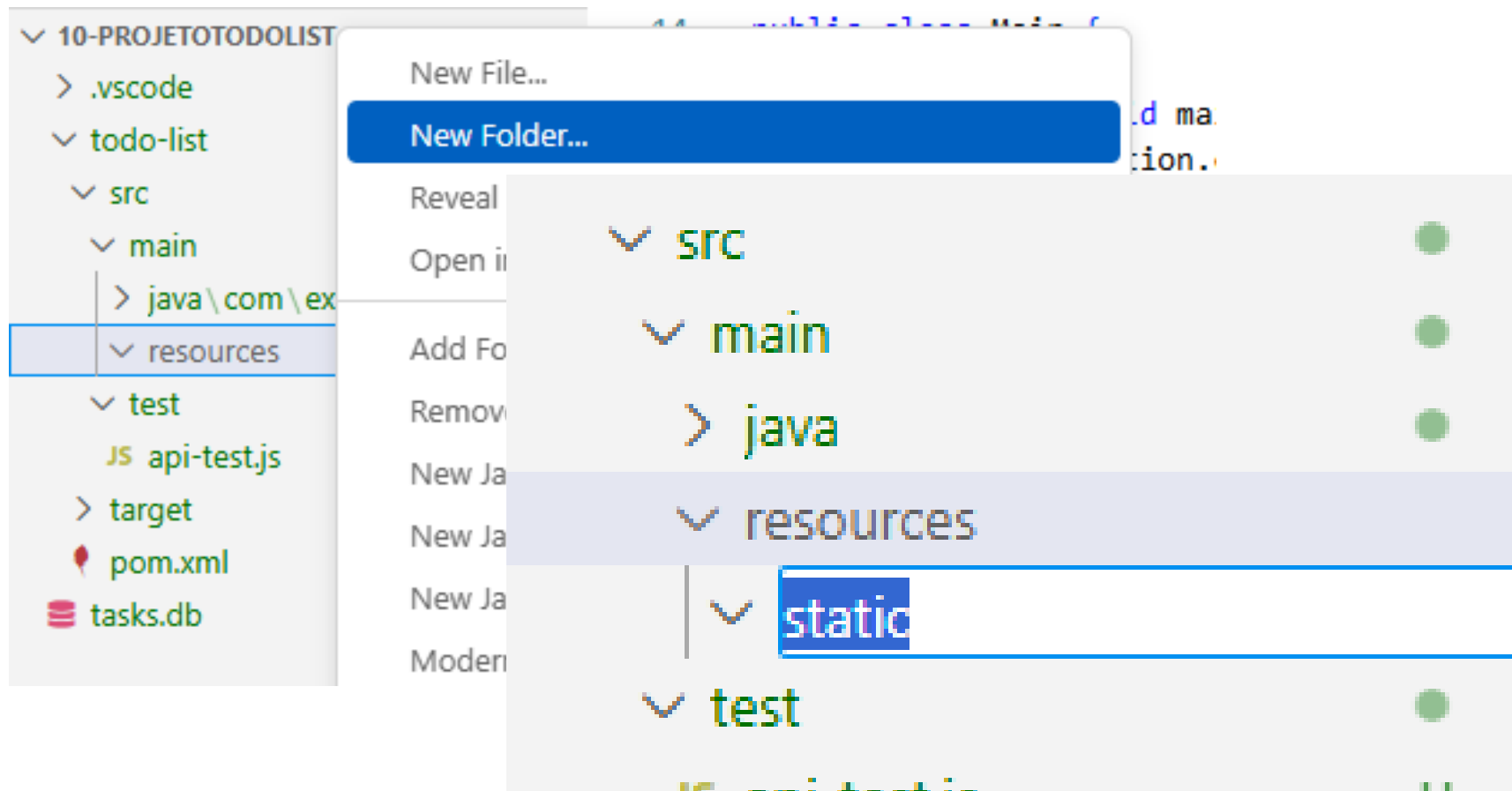
```
PS C:\Users\rllang\OneDrive\Documents\GitHub\JavaPoo\java-poo\10-ProjetoTodoList> & 'C:\Program Files\Java\openjdk-19.0.2\bin\java.exe' '@C:\Users\rllang\AppData\Local\Temp\cp_6e4uiwy8n97h3te6sphlf74m.argfile' 'com.example.Main'
Tabelas criadas/verif.
Admin padrão criado: username=admin, password=admin123
13:53:07.996 [main] INFO io.javalin.Javalin - Starting Javalin ...
13:53:08.000 [main] INFO org.eclipse.jetty.server.Server - jetty-11.0.25; built: 2025-03-13T00:15:57.301Z; git: a2e9fae3ad8320f2a713d4fa29bba356a99d1295; jvm
19.0.2+7-44
13:53:08.062 [main] INFO o.e.j.s.s.DefaultSessionIdManager - Session workerName=node0
13:53:08.075 [main] INFO o.e.j.server.handler.ContextHandler - Started o.e.j.s.ServletContextHandler@4c1f22f3{/,null,AVAILABLE}
13:53:08.125 [main] INFO o.e.jetty.server.AbstractConnector - Started ServerConnector@15f47664[HTTP/1.1, (http/1.1)]{0.0.0.0:7078}
13:53:08.150 [main] INFO org.eclipse.jetty.server.Server - Started Server@4d518b32[STARTING][11.0.25,sto=0] @899ms
13:53:08.150 [main] INFO io.javalin.Javalin -
```

API rodando!

```

// Executar
runTests().catch(err => log('Erro nos testes: ' + err));
Iniciando testes API...
1. Login admin...
6 Promise {<pending>}
2.5 Fetch finished loading: POST "URL".
Login admin: 200 - {"token":"a094fc9d-6a9c-483f-85fe-1c7e17e4e30f"}
2. Logout admin...
Fetch finished loading: PUT "http://localhost:7078/logout".
Logout admin: 200 - {"message":"Logout OK"}
3. Login admin novamente...
Login admin: 200 - {"token":"01c76d35-400a-4fc3-a4b7-428530363d21"}
4. Criar user teste...
Criar user: 201 - {"message":"Usuário criado"}
5. Listar users...
Listar users: 200 - [{"id":3,"username":"admin","password":null,"role":"ADMIN","token":null},{id:9,"username":"teste","password":null,"role":"USER","token":null}]
6. Login user teste...
Fetch finished loading: GET "http://localhost:7078/api/admin/users".
Login teste: 200 - {"token":"a04b7172-8893-4408-b7c6-7105f50f3422"}
7. Criar task...
Criar task: 201 - {"message":"Tarefa criada"}
8. Listar tasks...
Fetch finished loading: GET "http://localhost:7078/api/tasks".
Listar tasks: 200 - [{"id":7,"title":"Tarefa de teste","done":false,"userId":9}]
9. Atualizar task...
Atualizar task: 200 - {"message":"Tarefa atualizada"}
10. Deletar task...
Fetch finished loading: PUT "http://localhost:7078/api/tasks/1".
Fetch finished loading: DELETE "http://localhost:7078/api/tasks/2".
Deletar task: 200 - {"message":"Tarefa deletado"}
11. Logout user...
Logout user: 200 - {"message":"Logout OK"}
12. Deletar user teste...
Fetch finished loading: PUT "http://localhost:7078/logout".
Fetch finished loading: DELETE "http://localhost:7078/api/admin/users/teste".
Deletar user: 200 - {"message":"Usuário deletado"}
13. Logout admin final...
Fetch finished loading: PUT "http://localhost:7078/logout".
Logout admin: 200 - {"message":"Logout OK"}
Testes finalizados.
```

Testando a API



Criando uma pasta para o HTML

10-PROJETOTODOLIST

> .vscode

todo-list

src

main

java

resources \ static

test

JS api-test.js

New File...

New Folder...

src

main

java

resources \ static

<> index.html

test

EXPLORER

... index.html U X

OPEN EDITORS

X index.html todo-list/src\... U

10-PROJETOTODOLIST

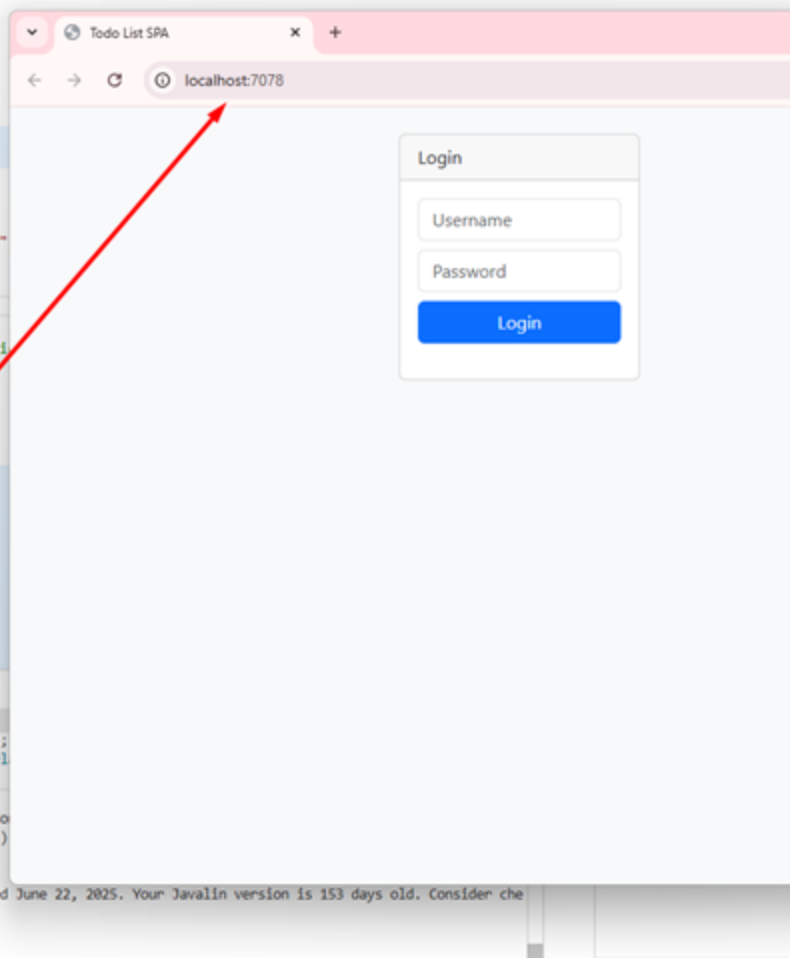
- .vscode
- todo-list
 - src
 - main
 - java\com\example
 - database
 - model
 - repository
 - Main.java U
 - resources\static
 - index.html U
 - test
 - api-test.js U
 - target
 - pom.xml U
 - tasks.db U

todo-list > src > main > resources > static > index.html > html

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>...
16 </head>
17 <body class="bg-light">
18
19     <div x-data="todoApp()" x-init="initApp()" x-cloak>
20
21 >     <nav class="navbar navbar-expand-lg navbar-dark bg-primary" x-show="isLoggedIn">...
51     </nav>
52
53     <div class="container mt-4">
54
55 >     <div x-show="message.text" class="alert alert-dismissible fade show" :class="'alert-' + message.type" role="alert">...
58     </div>
59
60 >     <div x-show="!isLoggedIn" class="row justify-content-center mt-5">...
81     </div>
82
83 >     <div x-show="isLoggedIn && view === 'tasks'">...
121     </div>
122
123 >     <div x-show="isLoggedIn && isAdmin && view === 'admin_users'">...
194     </div>
195
196 >     <div x-show="isLoggedIn && isAdmin && view === 'admin_tasks'">...
229     </div>
230
231     </div>
232 </div>
233
234 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
235
236 > <script>...
461 </script>
462 </body>
463 </html>
```



```
index.html U J Main.java 2, U X
todo-list > src > main > java > com > example > J Main.java > Main > main(String[])
17 import java.util.UUID;
18
19 public class Main {
20     public static void main(String[] args) {
21         SQLiteConnection.createTables();
22
23         // Cria admin padrão se não houver usuários
24         try { ...
25     } catch (Exception e) { ...
26
27     Javalin app = Javalin.create(config -> {
28         // Define o Content-Type padrão para todas as respostas
29         config.http.defaultContentType = "text/plain; charset=UTF-8"
30
31         // Configura arquivos estáticos se necessário
32         config.staticFiles.add("/static", Location.CLASSPATH);
33
34         // Se estiver usando arquivos estáticos ou templates, verifi
35         config.pvt.javaLangErrorHandler((res, error) -> {
36             res.setCharacterEncoding("UTF-8");
37         });
38     }) start(7078);
39
40     app.before("/api/*", ctx -> { ...
41     app.before("/api/admin/*", ctx -> { ...
42     app.post("/login", ctx -> { ...
43     app.put("/logout", ctx -> { ...
44     app.get("/api/admin/users", ctx -> { ...
45     app.post("/api/admin/users", ctx -> { ...
46     app.delete("/api/admin/users/{username}", ctx -> { ...
47     app.get("/api/admin/tasks", ctx -> { ...
48     app.get("/api/tasks", ctx -> { ...
49     app.post("/api/tasks", ctx -> { ...
50
51     OUTPUT DEBUG CONSOLE PORTS GITLENS
52
53     TERMINAL
54     PS C:\Users\rlang\OneDrive\Documents\GitHub\JavaPOO\java-poo\10-ProjetoTodoList> c:
55     -ProjetoTodoList'; & 'C:\Program Files\Java\openjdk-19.0.2\bin\java.exe' '@C:\Users\rl
56     example.Main'
57     16:36:45.406 [main] INFO io.javalin.Javalin - Javalin started in 175ms \o/
58     16:36:45.650 [main] INFO io.javalin.Javalin - Static file handler added: StaticFileCo
59     ess=false, aliasCheck=null, headers={Cache-Control-max-age=0}, mimeTypes={}, roles=[]
60     s/GitHub/JavaPOO/java-poo\10-ProjetoTodoList\todo-list\target\classes\static/"
61     16:36:45.657 [main] INFO io.javalin.Javalin - Listening on http://localhost:7078/
62     16:36:45.670 [main] INFO io.javalin.Javalin - You are running Javalin 6.7.0 (released June 22, 2025. Your Javalin version is 153 days old. Consider che
63     cking for a newer ve
64     Servidor rodando em http://localhost:7078
```



Inclua pasta
“static” na
configuração do
Javalin e remova
a rota “/”

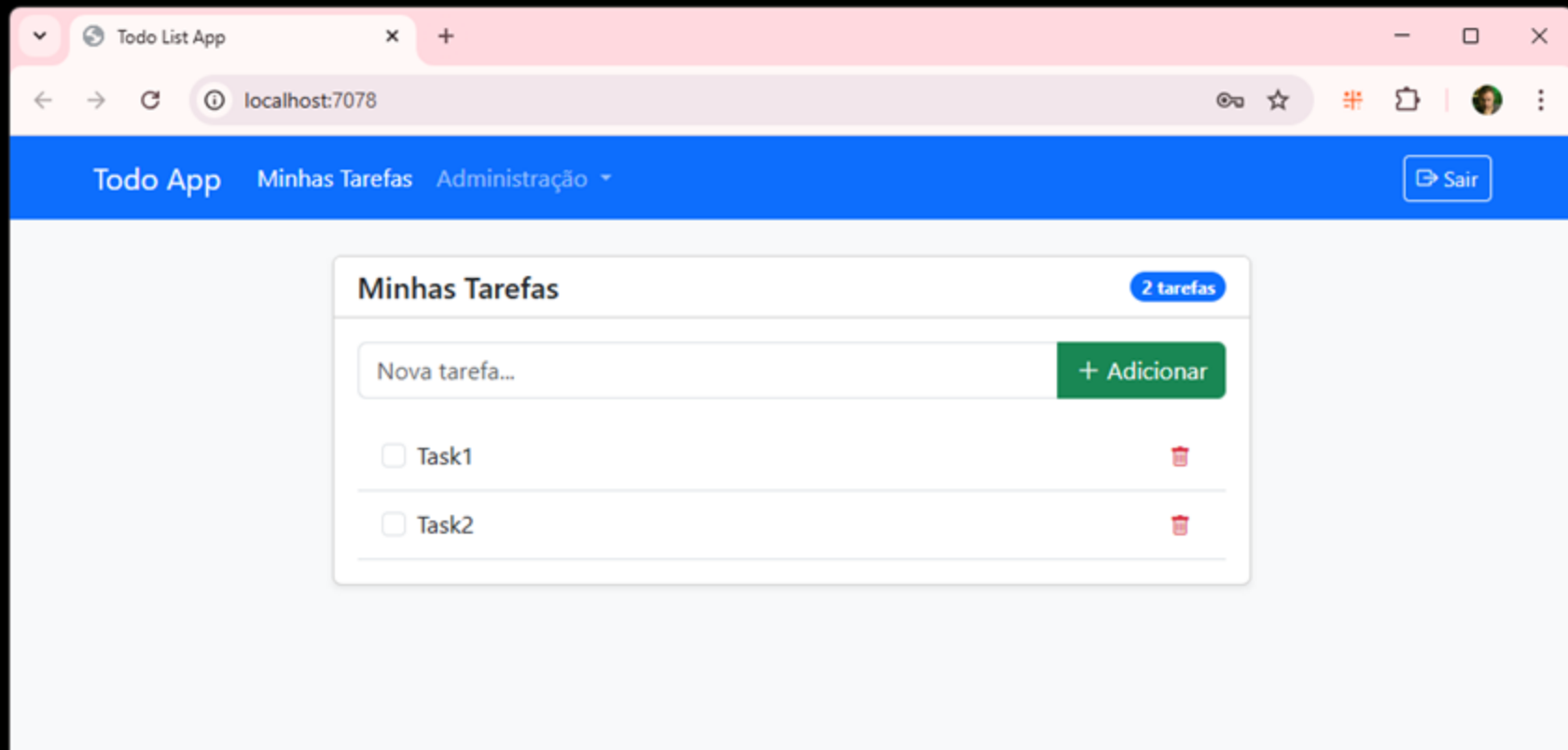


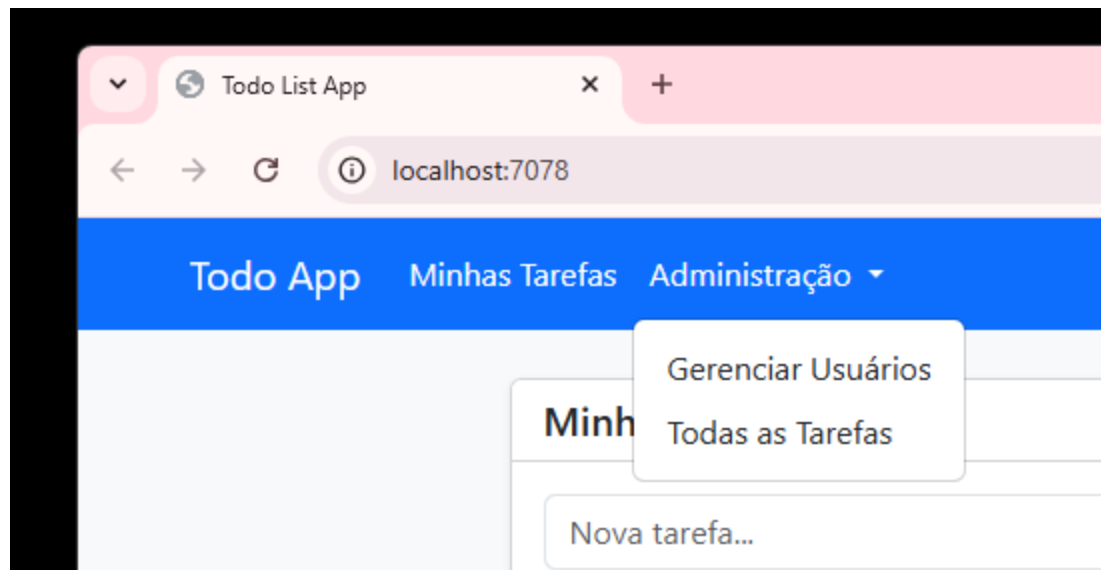
Login

Usuário

Senha

Entrar

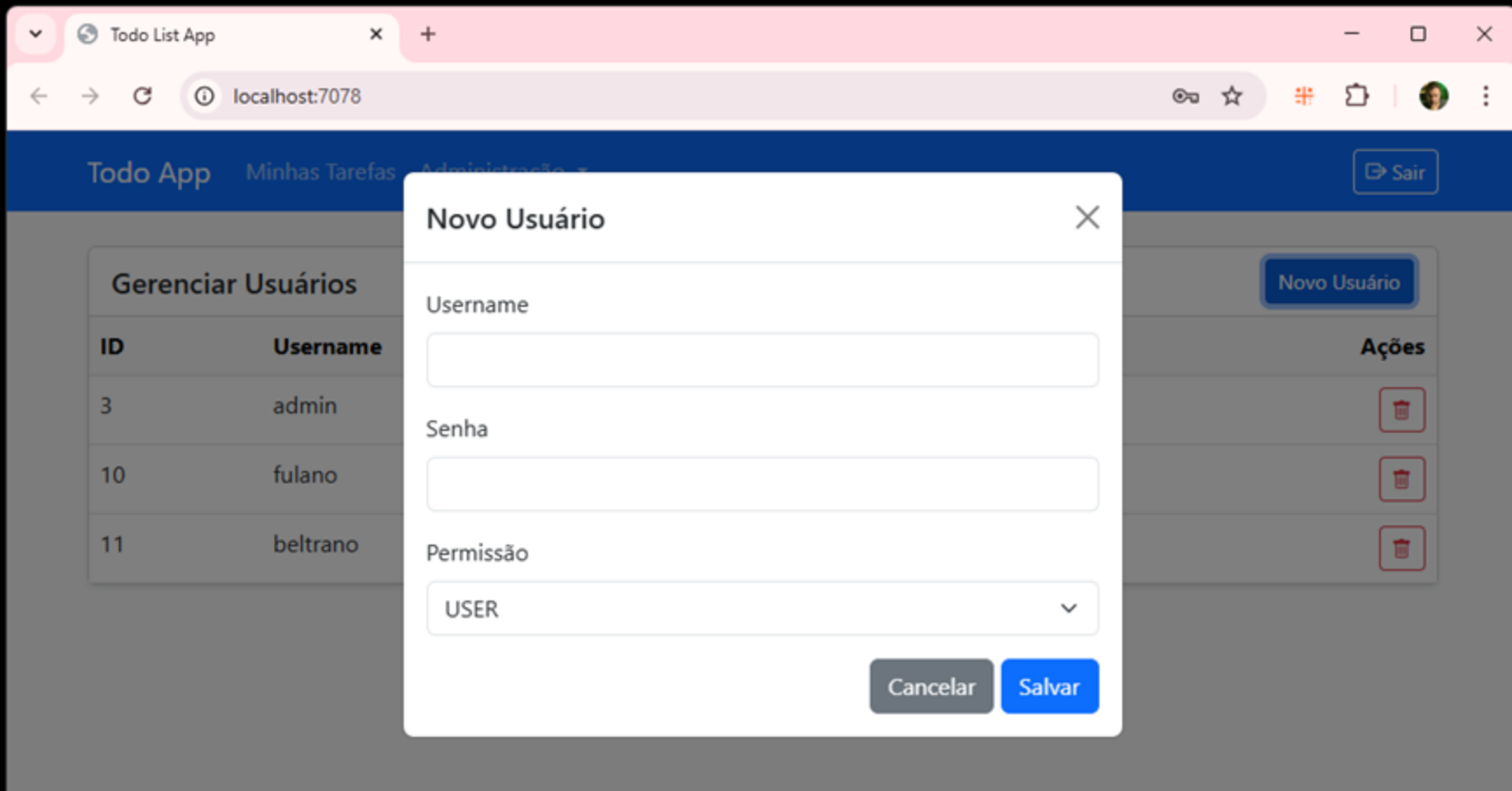




Gerenciar Usuários

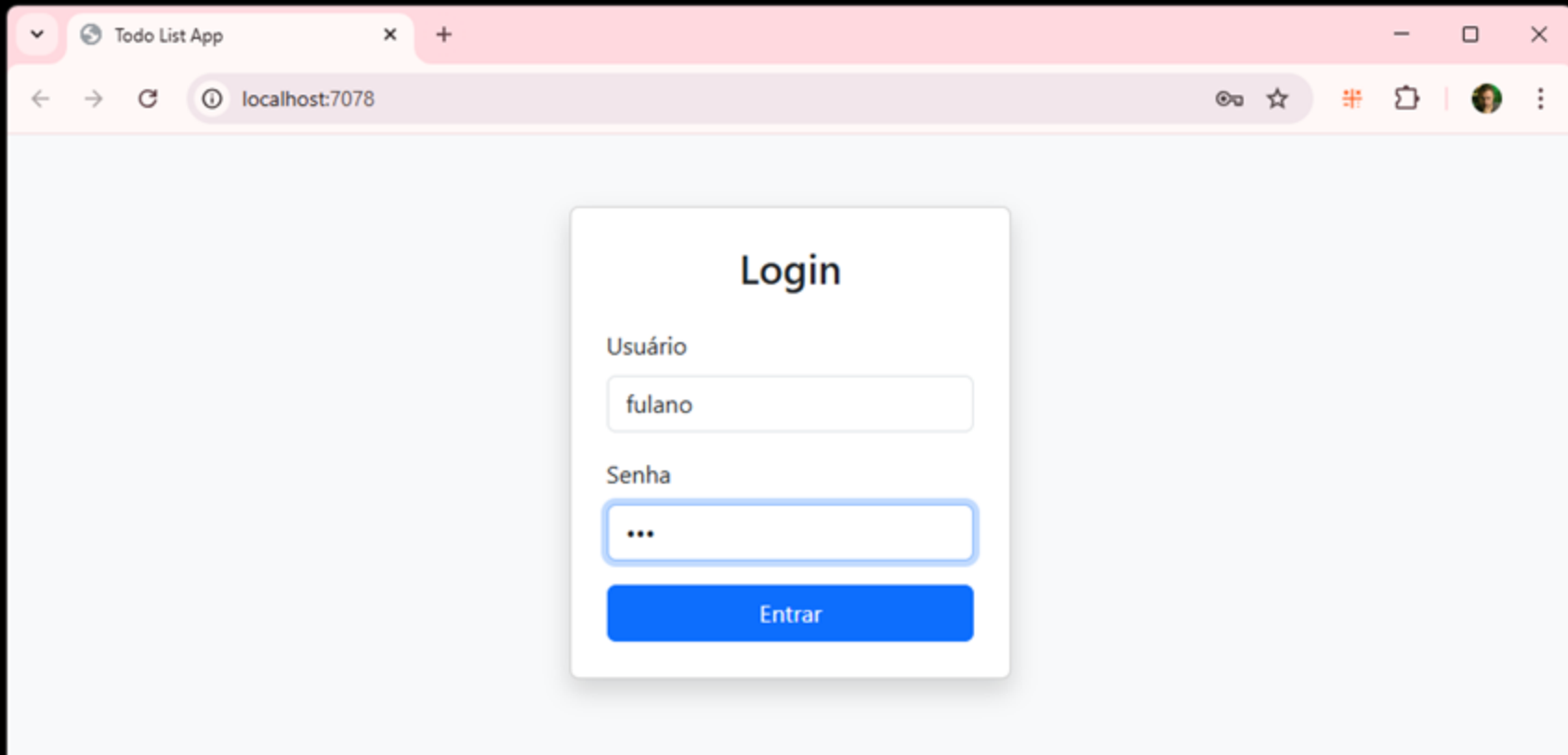
Novo Usuário

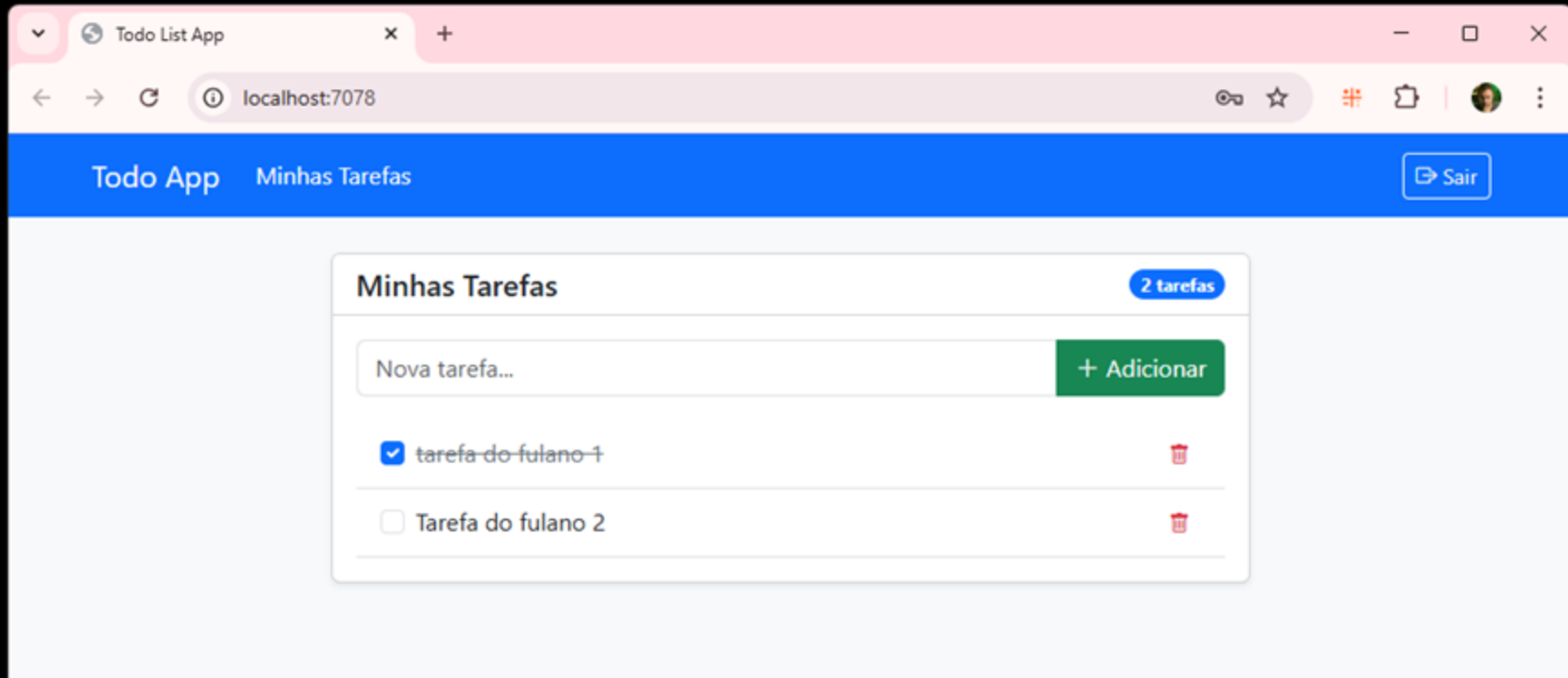
ID	Username	Role	Ações
3	admin	ADMIN	
10	fulano	USER	
11	beltrano	USER	



Todas as Tarefas (Visão Geral)

ID	Título	Status	User ID
8	Task1	Pendente	3
9	Task2	Pendente	3
13	tarefa do fulano 1	Concluído	10
14	Tarefa do fulano 2	Pendente	10
15	Tarefa do Beltrano 1	Pendente	11
16	Tarefa do Bel 2	Concluído	11
17	Tarefa do Bel três	Pendente	11





PROJETO

WebParkControl

Sumário

- Aplicação prática fullstack:
 - Fazer um sistema de controle de estacionamento no sqlite
 - Backend:
 - Implementar tabelas para fazer um sistema de controle de entrada e saída de veículos de um estacionamento
 - Implementar controle de acesso por token
 - Papéis: ADMIN e USER, com enum
 - Frontend:
 - Habilitar arquivos estáticos no Javalin para publicar assets e páginas HTML
 - Implementar uma interface dinâmica utilizando Vue.js

